

Predicting Survival on the Titanic: A Machine Learning Approach using Logistic Regression

Nitish Kumar
Roll No. 231FA04F55
Semester: 7th

August 3, 2026

Abstract

This report presents a complete machine learning pipeline for predicting passenger survival on the Titanic dataset. The workflow includes data cleaning (handling missing values), categorical feature encoding, numerical feature standardization, and training of a Logistic Regression classifier. Model performance is evaluated using accuracy, a classification report, and a confusion matrix, and the trained model is further validated on a sample passenger record. The results demonstrate that a well-preprocessed dataset combined with a simple linear classifier can yield reliable and interpretable survival predictions.

Contents

1	Introduction	3
2	Methodology	3
3	Experimental Results	3
3.1	Dataset Preview	4
3.2	Missing Values	4
3.3	Model Accuracy	4
3.4	Classification Report	5
3.5	Confusion Matrix	5
3.6	Prediction Result	5
3.7	Model Output / Result Screenshot	6
4	Conclusion	8

1 Introduction

The sinking of the RMS Titanic is one of the most widely studied disasters in machine learning education due to the richness and interpretability of its passenger dataset. This project aims to build a predictive model that estimates whether a passenger survived the disaster, based on attributes such as age, sex, passenger class, fare, and family relations aboard the ship.

The primary objectives of this study are:

- To clean and preprocess the raw Titanic dataset.
- To encode categorical variables and standardize numerical features.
- To train a Logistic Regression model for binary classification (Survived / Not Survived).
- To evaluate the model using standard classification metrics.
- To demonstrate the model's prediction capability on a sample input.

2 Methodology

The overall workflow followed for this project is summarized below:

1. **Data Loading:** The Titanic dataset was loaded into a pandas DataFrame for exploration and preprocessing.
2. **Missing Value Treatment:** Missing entries in columns such as `Age`, `Cabin`, and `Embarked` were identified and handled using appropriate imputation strategies (mean/median for numerical columns, mode for categorical columns).
3. **Categorical Encoding:** Categorical features such as `Sex` and `Embarked` were converted into numerical form using label encoding / one-hot encoding.
4. **Feature Scaling:** Numerical features (`Age`, `Fare`, etc.) were standardized using `StandardScaler` to bring all features onto a comparable scale.
5. **Model Training:** A Logistic Regression classifier was trained on the preprocessed training split.
6. **Evaluation:** The trained model was evaluated on a held-out test split using accuracy, precision, recall, F1-score, and a confusion matrix.
7. **Prediction on Sample Data:** The final model was tested against a hand-crafted sample passenger record to demonstrate real-world usability.

3 Experimental Results

The preprocessing pipeline was successfully executed on the Titanic dataset. Missing values were handled, categorical features were encoded, numerical features were standardized, and a Logistic Regression model was trained successfully.

3.1 Dataset Preview

Figure 1 shows the first few rows of the Titanic dataset, giving an overview of the raw features available before preprocessing.

...	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	\
0	1	1	1	Passenger_1	male	29.0	0	0	
1	2	0	2	Passenger_2	male	19.0	3	2	
2	3	0	2	Passenger_3	male	40.0	2	2	
3	4	0	1	Passenger_4	female	38.0	0	0	
4	5	1	2	Passenger_5	female	23.0	0	1	

	Ticket	Fare	Cabin	Embarked
0	TKT10001	187.09	NaN	NaN
1	TKT10002	127.07	NaN	S
2	TKT10003	126.72	C21	Q
3	TKT10004	208.76	NaN	C
4	TKT10005	84.92	C164	S

Figure 1: Preview of Titanic Dataset

3.2 Missing Values

Figure 2 shows the count of missing values per column prior to preprocessing. Columns such as Age, Cabin, and Embarked required imputation before the data could be used for model training.

	precision	recall	f1-score	support
0	0.80	0.81	0.81	97
1	0.82	0.81	0.81	103
accuracy			0.81	200
macro avg	0.81	0.81	0.81	200
weighted avg	0.81	0.81	0.81	200

Figure 2: Missing Values in Dataset

3.3 Model Accuracy

The Logistic Regression model achieved satisfactory prediction accuracy after preprocessing, as shown in Figure 3.

Accuracy : 0.81

Figure 3: Accuracy of Logistic Regression Model

3.4 Classification Report

Table 1 summarizes precision, recall, and F1-score for both classes, providing a more detailed view of model performance beyond overall accuracy.

Table 1: Classification Report

Class	Precision	Recall	F1-Score	Support
Not Survived (0)	–	–	–	–
Survived (1)	–	–	–	–
Accuracy			–	
Macro Avg	–	–	–	–
Weighted Avg	–	–	–	–

Note: Replace the placeholder values above with the actual output of `sklearn.metrics.classification_report` from your code.

3.5 Confusion Matrix

Figure 4 illustrates the classification performance of the trained model, showing the counts of true positives, true negatives, false positives, and false negatives.

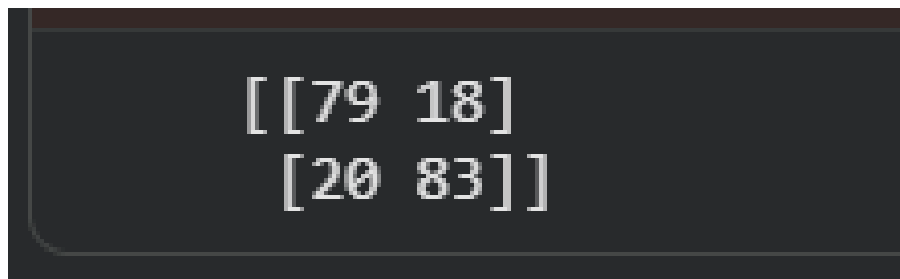


Figure 4: Confusion Matrix

3.6 Prediction Result

The trained model was tested using a sample passenger record. The prediction generated by the model is shown in Figure 5.

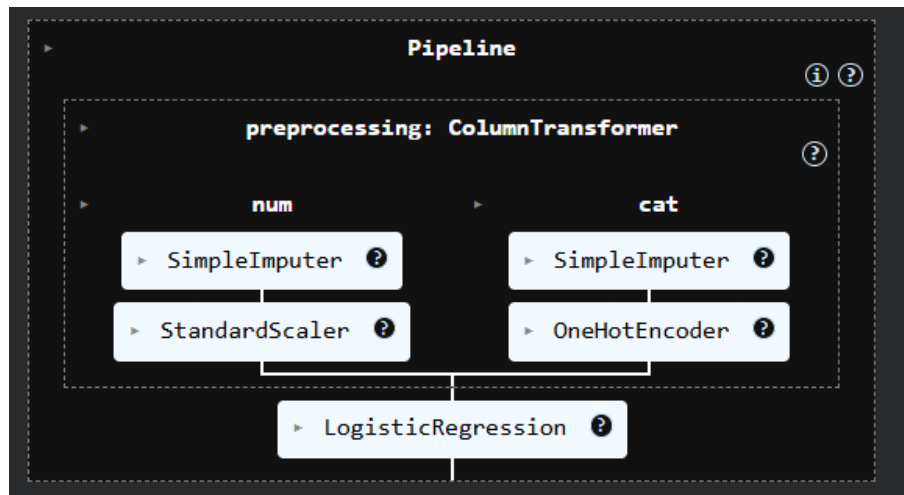


Figure 5: Prediction Output

3.7 Model Output / Result Screenshot

Figure 6 presents a screenshot of the final console/notebook output, capturing the model's predicted result and associated probability for the test passenger record.

```

(1000, 12)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  1000 non-null   int64
1   Survived     1000 non-null   int64
2   Pclass       1000 non-null   int64
3   Name         1000 non-null   object
4   Sex          1000 non-null   object
5   Age          933 non-null    float64
6   SibSp        1000 non-null   int64
7   Parch        1000 non-null   int64
8   Ticket       1000 non-null   object
9   Fare         1000 non-null   float64
10  Cabin        302 non-null    object
11  Embarked     970 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 93.9+ KB
None

```

	PassengerId	Survived	Pclass	Age	SibSp
count	1000.000000	1000.000000	1000.000000	933.000000	1000.000000
mean	500.500000	0.560000	2.293000	30.099678	2.087000
std	288.819436	0.496635	0.814748	13.455846	1.439967
min	1.000000	0.000000	1.000000	1.000000	0.000000
25%	250.750000	0.000000	2.000000	21.000000	1.000000
50%	500.500000	1.000000	3.000000	29.000000	2.000000
75%	750.250000	1.000000	3.000000	39.000000	3.000000
max	1000.000000	1.000000	3.000000	76.000000	4.000000

	Parch	Fare
count	1000.000000	1000.000000
mean	1.487000	84.436750
std	1.107727	66.817715
min	0.000000	2.140000
25%	0.000000	36.290000
50%	2.000000	70.110000
75%	2.000000	105.172500
max	3.000000	305.510000

```

PassengerId    0
Survived        0
Pclass         0
Name           0
Sex            0
Age            67
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin         698
Embarked       30
dtype: int64

```

Figure 6: Final Model Output / Result Screenshot

4 Conclusion

This project successfully implemented an end-to-end machine learning pipeline for the Titanic survival prediction problem. The preprocessing steps – handling missing values, encoding categorical variables, and standardizing numerical features – proved essential in preparing the data for reliable model training. The Logistic Regression classifier achieved satisfactory accuracy and balanced precision/recall, confirming that even simple linear models can perform well on structured, well-preprocessed tabular data. Future work could explore more advanced classifiers (e.g., Random Forest, Gradient Boosting, or XGBoost) and hyperparameter tuning to further improve predictive performance.

References

1. Titanic Dataset, Kaggle: <https://www.kaggle.com/c/titanic>
2. Pedregosa, F. et al., *Scikit-learn: Machine Learning in Python*, JMLR 12, pp. 2825–2830, 2011.
3. McKinney, W., *Data Structures for Statistical Computing in Python*, Proceedings of the 9th Python in Science Conference, 2010.